

Inserções concorrentes em listas encadeadas com OpenMP

Matheus Marinho

13 de setembro de 2026

1. Objetivo e organização

O programa cria `N` tarefas OpenMP. Cada tarefa produz um nó contendo seu número, escolhe pseudoaleatoriamente uma lista e o insere no início dela. Como várias tarefas podem ler e modificar simultaneamente o ponteiro `inicio`, a atualização precisa ser mutuamente exclusiva: sem proteção, duas tarefas poderiam ler o mesmo início, gravar sucessores diferentes e uma delas sobrescrever a atualização da outra, perdendo um nó.

Foram implementadas e medidas três versões:

Versão	Listas	Proteção
comparação	2	um <code>critical</code> sem nome para ambas
solução específica	2	<code>critical(lista_zero)</code> e <code>critical(lista_um)</code>
generalização	definida pelo usuário	um <code>omp_lock_t</code> por lista

As listas são recursos compartilhados. Uma tarefa pode ser executada por qualquer thread da equipe; o OpenMP não fixa permanentemente uma lista ou uma tarefa a uma thread. O importante é que tarefas destinadas a listas distintas possam progredir simultaneamente quando a forma de sincronização permitir.

2. Criação das tarefas e escolha aleatória

Somente uma thread cria as `N` tarefas, evitando que cada thread repita todo o laço. `firstprivate(i)` dá a cada tarefa uma cópia do seu identificador, enquanto as listas e seus mecanismos de proteção permanecem compartilhados. O `taskgroup` garante que todas as inserções acabaram antes da validação ou liberação da memória.

```
#pragma omp parallel default(none) shared(/* dados compartilhados */)
{
    #pragma omp single
    {
        #pragma omp taskgroup
        {
            for (long long i = 0; i < insercoes; i++) {
                #pragma omp task firstprivate(i)
                {
                    int destino = escolher_lista(i, semente, quantidade);
                    /* alocação, aquisição da proteção e inserção */
                }
            }
        }
    }
}
```

A função `escolher_lista` mistura o identificador único da tarefa com uma semente e calcula o resto pelo número de listas. Isso fornece uma distribuição pseudoaleatória reproduzível sem compartilhar o estado de `rand()`. Portanto, a própria escolha não introduz uma condição de corrida e as três versões recebem a mesma sequência de destinos quando usam a mesma semente.

3. Duas listas com regiões críticas nomeadas

A inserção altera dois campos relacionados, `inicio` e `tamanho`, sob a mesma proteção:

```
static void inserir_sem_protecao(Lista *lista, No *novo)
{
    novo->proximo = lista->inicio;
    lista->inicio = novo;
    lista->tamanho++;
}
```

Um `critical` sem nome usa o mesmo lock implícito que todos os outros `critical` sem nome do programa. Consequentemente, uma inserção na lista 0 bloqueia outra na lista 1 mesmo sem haver conflito de dados:

```
#pragma omp critical
```

```
inserir_sem_protecao(&listas[destino], novo);
```

Com duas listas conhecidas ao compilar, dois nomes resolvem a serialização desnecessária:

```
if (destino == 0) {
    #pragma omp critical(lista_zero)
    inserir_sem_protecao(&listas[0], novo);
} else {
    #pragma omp critical(lista_um)
    inserir_sem_protecao(&listas[1], novo);
}
```

Regiões com o mesmo nome são mutuamente exclusivas; nomes diferentes representam domínios de sincronização diferentes. Assim, duas tarefas destinadas à mesma lista continuam serializadas, preservando sua integridade, enquanto tarefas em listas distintas podem executar as pequenas atualizações ao mesmo tempo.

4. Generalização e necessidade de locks explícitos

O nome de uma região `critical` é um identificador escrito no código-fonte. Ele não pode ser um índice, uma string construída durante a execução ou um elemento de vetor. Seria possível escrever manualmente um grande `switch` com muitos nomes, mas isso imporaria um limite decidido na compilação e não atenderia a uma quantidade arbitrária informada pelo usuário.

A generalização aloca um vetor de listas e outro com exatamente um lock por lista. O índice pseudoaleatório seleciona tanto a lista como sua proteção:

```
int destino = escolher_lista(i, semente, quantidade);
omp_set_lock(&locks[destino].valor);
inserir_sem_protecao(&listas[destino], novo);
omp_unset_lock(&locks[destino].valor);
```

O ciclo de vida de cada lock é explícito. `omp_init_lock` deve ser chamado antes de qualquer tarefa usá-lo; `omp_set_lock` adquire; `omp_unset_lock` libera; e `omp_destroy_lock` encerra o objeto depois do `taskgroup`. `destroy` não substitui `free`: ele libera eventuais recursos do runtime associados ao lock, enquanto `free` libera os vetores alocados. Embora o sistema operacional recupere memória ao terminar o processo, omitir essas operações é uma prática incorreta, prejudica ferramentas de diagnóstico e falha se a rotina for reutilizada por um programa que continua executando.

5. Falso compartilhamento dos locks

Locks logicamente independentes também podem interferir pelo hardware. Se vários `omp_lock_t` pequenos e contíguos ocuparem a mesma linha de cache, uma thread que modifica o lock da lista 0 invalida a linha inteira nos caches dos outros núcleos. Outra thread que modifica o lock da lista 1 precisa então transferir novamente a mesma linha, apesar de as duas listas serem independentes. Esse tráfego de coerência é chamado **falso compartilhamento**.

Na máquina do experimento, a linha de cache possui 64 bytes. Por isso o programa envolve cada lock em uma estrutura com 64 bytes adicionais:

```
#define CACHE_LINE 64
typedef struct {
    omp_lock_t valor;
    unsigned char separacao[CACHE_LINE];
} LockSeparado;
```

O início de dois locks consecutivos fica separado por mais de 64 bytes, impedindo que ambos caiam na mesma linha de 64 bytes. Os cabeçalhos das listas recebem separação equivalente, pois permitir inserções simultâneas e depois colocar os ponteiros `inicio` na mesma linha de cache recriaria o mesmo gargalo. O valor 64 é uma hipótese documentada da plataforma medida; a linguagem C e o OpenMP não oferecem um tamanho universal de linha de cache.

6. Verificação de integridade

Depois que o `taskgroup` termina, cada lista é percorrida sem concorrência. O programa compara três valores: a soma dos campos `tamanho`, a quantidade real de nós alcançados pelos ponteiros e `N` menos eventuais falhas de alocação. O estado só é marcado `OK` se os três coincidirem. Isso detecta o sintoma típico da corrida na inserção, em que um encadeamento é sobrescrito e nós deixam de ser alcançáveis.

Uma falha de `malloc` é contada com `atomic`, pois o contador de falhas também é compartilhado. O nó só entra na região crítica depois de ser alocado; dessa forma, o lock protege apenas a alteração estrutural curta da lista.

7. Resultados e análise de desempenho

O código foi compilado sem avisos com:

```
gcc -O2 -Wall -Wextra -std=c11 -fopenmp listas_insercoes.c -o listas_insercoes.exe
```

Foram realizadas cinco execuções com 200.000 inserções, oito threads e oito listas na versão generalizada. Todas as 15 combinações de versão e repetição armazenaram 200.000 nós, não tiveram falha de alocação e terminaram com integridade OK.

Versão	Tempos das 5 execuções (s)	Mediana (s)
2 listas, <code>critical</code> comum	0,576; 0,581; 0,588; 0,560; 0,577	0,577
2 listas, <code>critical</code> nomeado	0,560; 0,571; 0,546; 0,563; 0,583	0,563
8 listas, locks explícitos	0,408; 0,426; 0,439; 0,438; 0,362	0,426

Na mediana, nomear as duas regiões reduziu cerca de 2,4% do tempo em relação ao `critical` comum, diferença pequena demais para afirmar um ganho estável. A possibilidade de duas inserções simultâneas existe, mas o trecho protegido contém apenas duas atualizações de ponteiro/contador; criação e escalonamento das tarefas, `malloc`, `free` e tráfego de cache continuam presentes. Além disso, dois locks permitem mais concorrência, não garantem menor tempo: em algumas repetições o nomeado foi mais lento por variação de escalonamento e custo de coerência. Portanto, a vantagem principal é remover uma dependência falsa; o efeito temporal depende da carga e deve ser medido.

Com oito listas, os locks explícitos obtiveram mediana 26,2% menor que o `critical` comum de duas listas. Essa comparação também muda a quantidade de listas e, conseqüentemente, reduz a probabilidade de contenção; ela demonstra a escalabilidade da generalização, mas não isola somente o custo da API de locks.

8. Conclusão

As inserções são corretas porque toda alteração em um cabeçalho ocorre enquanto a proteção exclusiva daquela lista está adquirida e porque a memória permanece válida até o fim das tarefas. Para duas listas fixas, regiões críticas nomeadas expressam com clareza duas exclusões mútuas independentes. Para uma quantidade conhecida apenas em execução, nomes de diretivas não podem ser gerados dinamicamente; um vetor de `omp_lock_t` fornece exatamente o mapeamento necessário entre índice e lock. Separar fisicamente locks e cabeçalhos evita que a coerência de cache volte a acoplar estruturas que o algoritmo tornou independentes.

9. Código-fonte completo

listas_insercoes.c · 311 linhas

```
1 /*
2  * Tarefa 9 - tarefas, listas encadeadas, critical nomeado e locks
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c11 -fopenmp listas_insercoes.c -o listas_insercoes.exe
6  *
7  * Executar:
8  * listas_insercoes.exe [insercoes] [threads] [listas] [semente]
9  */
10
11 #include <errno.h>
12 #include <limits.h>
13 #include <omp.h>
14 #include <stdint.h>
15 #include <stdio.h>
16 #include <stdlib.h>
17
18 #define CACHE_LINE 64
19
20 typedef struct No {
21     long long valor;
22     struct No *proximo;
23 } No;
24
25 /* A separacao evita que os cabecalhos de duas listas disputem a mesma linha. */
26 typedef struct {
27     No *inicio;
28     size_t tamanho;
29     unsigned char separacao[CACHE_LINE];
30 } Lista;
31
32 /* Instancias consecutivas ficam a mais de uma linha de cache de distancia. */
33 typedef struct {
34     omp_lock_t valor;
35     unsigned char separacao[CACHE_LINE];
36 } LockSeparado;
37
38 _Static_assert(sizeof(LockSeparado) > CACHE_LINE,
39                "locks consecutivos precisam ficar separados");
40
41 typedef struct {
42     double segundos;
43     long long inseridos;
44     long long falhas_alocacao;
45     int valido;
46 } Resultado;
47
48 typedef enum {
49     CRITICAL_COMUM,
50     CRITICAL_NOMEADO
51 } ModoDuasListas;
52
53 /* Misturador deterministico: cada tarefa tem seu proprio estado, sem rand(). */
54 static uint64_t misturar(uint64_t x)
55 {
56     x += UINT64_C(0x9e3779b97f4a7c15);
57     x = (x ^ (x >> 30)) * UINT64_C(0xbf58476d1ce4e5b9);
58     x = (x ^ (x >> 27)) * UINT64_C(0x94d049bb133111eb);
59     return x ^ (x >> 31);
60 }
61
62 static int escolher_lista(long long tarefa, unsigned int semente, int quantidade)
63 {
64     uint64_t estado = (uint64_t)tarefa ^ ((uint64_t)semente << 32);
65     return (int)(misturar(estado) % (uint64_t)quantidade);
66 }
67
68 /* Deve ser chamada somente enquanto a protecao da lista estiver adquirida. */
69 static void inserir_sem_protecao(Lista *lista, No *novo)
70 {
71     novo->proximo = lista->inicio;
72     lista->inicio = novo;
73     lista->tamanho++;
74 }
75
76 static void inserir_critical_comum(Lista listas[2], int destino, No *novo)
77 {
78     /* O critical sem nome usa o mesmo lock implicito para as duas listas. */
79     #pragma omp critical
80     inserir_sem_protecao(&listas[destino], novo);
81 }
82
```

```

83 static void inserir_critical_nomeado(Lista listas[2], int destino, No *novo)
84 {
85     /* Os nomes representam locks implícitos diferentes e conhecidos em compilação. */
86     if (destino == 0) {
87         #pragma omp critical(lista_zero)
88         inserir_sem_protecao(&listas[0], novo);
89     } else {
90         #pragma omp critical(lista_um)
91         inserir_sem_protecao(&listas[1], novo);
92     }
93 }
94
95 static void liberar_listas(Lista *listas, int quantidade)
96 {
97     for (int i = 0; i < quantidade; i++) {
98         No *atual = listas[i].inicio;
99         while (atual != NULL) {
100             No *seguinte = atual->proximo;
101             free(atual);
102             atual = seguinte;
103         }
104     }
105 }
106
107 static int validar_listas(const Lista *listas, int quantidade,
108                          long long esperado, long long *encontrados)
109 {
110     long long total_cabecalhos = 0;
111     long long total_percorrido = 0;
112     int valido = 1;
113
114     for (int i = 0; i < quantidade; i++) {
115         size_t percorridos = 0;
116         for (const No *atual = listas[i].inicio;
117              atual != NULL; atual = atual->proximo) {
118             percorridos++;
119         }
120         if (percorridos != listas[i].tamanho) {
121             valido = 0;
122         }
123         total_cabecalhos += (long long)listas[i].tamanho;
124         total_percorrido += (long long)percorridos;
125     }
126
127     *encontrados = total_percorrido;
128     return valido && total_cabecalhos == esperado &&
129            total_percorrido == esperado;
130 }
131
132 static Resultado executar_duas_listas(long long insercoes,
133                                       unsigned int semente,
134                                       ModoDuasListas modo)
135 {
136     Lista listas[2] = {0};
137     long long falhas = 0;
138     double inicio = omp_get_wtime();
139
140     #pragma omp parallel default(none) \
141         shared(listas, falhas, insercoes, semente, modo)
142     {
143         #pragma omp single
144         {
145             #pragma omp taskgroup
146             {
147                 for (long long i = 0; i < insercoes; i++) {
148                     #pragma omp task firstprivate(i)
149                     {
150                         int destino = escolher_lista(i, semente, 2);
151                         No *novo = malloc(sizeof(*novo));
152
153                         if (novo == NULL) {
154                             #pragma omp atomic update
155                             falhas++;
156                         } else {
157                             novo->valor = i;
158                             if (modo == CRITICAL_NOMEADO) {
159                                 inserir_critical_nomeado(listas, destino, novo);
160                             } else {
161                                 inserir_critical_comum(listas, destino, novo);
162                             }
163                         }
164                     }
165                 }
166             }
167         }
168     }
169
170     Resultado r;
171     r.segundos = omp_get_wtime() - inicio;

```

```

172     r.falhas_alocacao = falhas;
173     r.valido = validar_listas(listas, 2, insercoes - falhas, &r.inseridos);
174     liberar_listas(listas, 2);
175     return r;
176 }
177
178 static Resultado executar_varias_listas(long long insercoes,
179                                         unsigned int semente,
180                                         int quantidade)
181 {
182     Lista *listas = calloc((size_t)quantidade, sizeof(*listas));
183     LockSeparado *locks = malloc((size_t)quantidade * sizeof(*locks));
184     Resultado r = {0.0, 0, 0, 0};
185     long long falhas = 0;
186
187     if (listas == NULL || locks == NULL) {
188         free(listas);
189         free(locks);
190         r.falhas_alocacao = insercoes;
191         return r;
192     }
193
194     for (int indice = 0; indice < quantidade; indice++) {
195         omp_init_lock(&locks[indice].valor);
196     }
197
198     double inicio = omp_get_wtime();
199     #pragma omp parallel default(none) \
200         shared(listas, locks, falhas, insercoes, semente, quantidade)
201     {
202         #pragma omp single
203         {
204             #pragma omp taskgroup
205             {
206                 for (long long i = 0; i < insercoes; i++) {
207                     #pragma omp task firstprivate(i)
208                     {
209                         int destino = escolher_lista(i, semente, quantidade);
210                         No *novo = malloc(sizeof(*novo));
211
212                         if (novo == NULL) {
213                             #pragma omp atomic update
214                             falhas++;
215                         } else {
216                             novo->valor = i;
217                             omp_set_lock(&locks[destino].valor);
218                             inserir_sem_protecao(&listas[destino], novo);
219                             omp_unset_lock(&locks[destino].valor);
220                         }
221                     }
222                 }
223             }
224         }
225     }
226     r.segundos = omp_get_wtime() - inicio;
227
228     r.falhas_alocacao = falhas;
229     r.valido = validar_listas(listas, quantidade,
230                               insercoes - falhas, &r.inseridos);
231
232     /* destroy libera recursos do runtime; free libera os vetores dinamicos. */
233     for (int indice = 0; indice < quantidade; indice++) {
234         omp_destroy_lock(&locks[indice].valor);
235     }
236     liberar_listas(listas, quantidade);
237     free(locks);
238     free(listas);
239     return r;
240 }
241
242 static int ler_ll(const char *texto, long long minimo, long long *valor)
243 {
244     char *fim;
245     long long convertido;
246
247     errno = 0;
248     convertido = strtoll(texto, &fim, 10);
249     if (errno != 0 || *texto == '\0' || *fim != '\0' || convertido < minimo) {
250         return 0;
251     }
252     *valor = convertido;
253     return 1;
254 }
255
256 static void imprimir_resultado(const char *nome, Resultado r)
257 {
258     printf("%-27s %12.6f %12lld %8lld  %s\n",
259           nome, r.segundos, r.inseridos, r.falhas_alocacao,
260           r.valido ? "OK" : "FALHOU");

```

```

261 }
262
263 int main(int argc, char **argv)
264 {
265     long long insercoes = 200000;
266     long long threads_lido = 8;
267     long long listas_lido = 8;
268     long long semente_lida = 2026;
269
270     if (argc > 5 ||
271         (argc > 1 && !ler_ll(argv[1], 1, &insercoes)) ||
272         (argc > 2 && !ler_ll(argv[2], 1, &threads_lido)) ||
273         (argc > 3 && !ler_ll(argv[3], 1, &listas_lido)) ||
274         (argc > 4 && !ler_ll(argv[4], 0, &semente_lida)) ||
275         threads_lido > INT_MAX || listas_lido > INT_MAX ||
276         semente_lida > UINT_MAX) {
277         fprintf(stderr,
278             "Uso: %s [insercoes>=1] [threads>=1] [listas>=1] [semente]\n",
279             argv[0]);
280         return EXIT_FAILURE;
281     }
282
283     int threads = (int)threads_lido;
284     int quantidade_listas = (int)listas_lido;
285     unsigned int semente = (unsigned int)semente_lida;
286
287     omp_set_dynamic(0);
288     omp_set_num_threads(threads);
289
290     printf("Tarefa 9 - insercoes concorrentes em listas encadeadas\n");
291     printf("N: %lld | threads: %d | listas na generalizacao: %d | semente: %u\n\n",
292         insercoes, threads, quantidade_listas, semente);
293     printf("Versao          Tempo (s)    Inseridos    Falhas    Integridade\n");
294
295     Resultado comum = executar_duas_listas(insercoes, semente, CRITICAL_COMUM);
296     Resultado nomeado = executar_duas_listas(insercoes, semente,
297         CRITICAL_NOMEADO);
298     Resultado locks = executar_varias_listas(insercoes, semente,
299         quantidade_listas);
300
301     imprimir_resultado("2 listas: critical comum", comum);
302     imprimir_resultado("2 listas: critical nomeado", nomeado);
303     imprimir_resultado("N listas: locks explicitos", locks);
304
305     if (comum.segundos > 0.0 && nomeado.segundos > 0.0) {
306         printf("\nRazao comum/nomeado: %.3fx\n",
307             comum.segundos / nomeado.segundos);
308     }
309     return (comum.valido && nomeado.valido && locks.valido)
310         ? EXIT_SUCCESS : EXIT_FAILURE;
311 }

```